Zeitnetz Generator

Full Technical Report

Algorithmic Replication of Lachenmann's Mouvement (– vor der Erstarrung) Time-Grid Generation

---

Based on: Luís Antunes Pena, *Klangnetz und Klangfarbe in Helmut Lachenmanns Mouvement (– vor der Erstarrung)*, Diplomarbeit, 2004

Implementation by: Paulo de Assis, in collaboration with Claude (Anthropic), March 2026

Source file: `zeitnetz_pipeline.py` (~2270 lines of Python)

Dependencies: Python 3.9, music21

Table of Contents

1. Project Overview

This project replicates, in a single Python pipeline, the complete algorithmic process by which Helmut Lachenmann generated the temporal structure (Zeitnetz — "time-net") of his orchestral work *Mouvement (– vor der Erstarrung)* (1982–84). The reconstruction follows Luís Antunes Pena's detailed musicological analysis, which reverse-engineered Lachenmann's process from the sketches and identified five compositional stages, originally realised in IRCAM's OpenMusic environment.

The pipeline takes three inputs — a 12-note pitch row, a permutation pattern, and a duration list — and produces, through a chain of deterministic transformations, the complete Zeitnetz with 75 sound families, variable time signatures, and the final duration-as-count transformation. All intermediate and final results are exported as MusicXML files for verification in standard notation software.

Default Inputs (Lachenmann's *Mouvement*)

Parameter	Values
Pitch row	1 11 0 8 9 3 6 4 2 10 5 7 (cis h c gis a dis fis e d ais f g)
Permutation pattern	1 5 0 6 2 7 11 8 3 10 4 9
Duration list	–11 6 9 7 6 6 4 3 10 6 3 1 10 (13 values; first is negative = initial rest)

Pitch Naming Convention

The pipeline uses German pitch names throughout, following Pena's analysis:

PC	0	1	2	3	4	5	6	7	8	9	10	11
Name	c	cis	d	dis	e	f	fis	g	gis	a	ais	h

2. Stage 1 — Row Generation

OM patch: "1-erzeugt-reihe" — **Function:** `run_stage1()` — **Export:** `zeitnetz_stage1.musicxml`

2.1 Permutation Matrix

The foundation of the entire system is a 12×12 permutation matrix. Starting from the identity row [0, 1, 2, ..., 11], each subsequent row is generated by applying the permutation pattern as an index reordering of the previous row:

```
Row 0 = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
Row k = [Row_{k-1}[perm[0]], Row_{k-1}[perm[1]], ..., Row_{k-1}[perm[11]]]
```

With the default permutation pattern [1, 5, 0, 6, 2, 7, 11, 8, 3, 10, 4, 9], this produces a 144-element structure that encodes all pitch and rhythmic relationships in the piece.

2.2 Onset Computation

The 13-element duration list is converted to 12 cumulative onset addresses. The first value (–11) indicates an initial rest of 11 units; its absolute value becomes the first onset. Subsequent onsets are cumulative sums:

```
onsets[0] = |duration_list[0]| = 11
onsets[i] = onsets[i-1] + duration_list[i]   (for i = 1..11)

Result: [11, 17, 26, 33, 39, 45, 49, 52, 62, 68, 71, 72]
```

2.3 Rhythm Row Derivation

The rhythm row is derived by mapping the pitch row through the flattened permutation matrix using the computed onsets as addresses: (1) Flatten the 12×12 matrix into a 144-element sequence. (2) For each pitch `pitch_row[i]`, look up `flattened[onsets[i]]` to get the target position. (3) Place the pitch at that position in the rhythm row.

Result: f c dis fis gis e g h d a ais cis

2.4 Permutation Generation

Both the pitch row and the rhythm row are then permuted 12 times using the permutation matrix, yielding **12 pitch permutations** (12×12 = 144 pitch values) and **12 rhythm permutations** (12×12 = 144 rhythm values). These two 144-element sequences form the raw material for Stage 2.

3. Stage 2 — Zeitnetz Version 1

OM patch: "2 - Zeitnetz Version 1" — **Function:** `run_stage2()` — **Export:** `zeitnetz_stage2.musicxml`

3.1 Circular Scanning Algorithm

Stage 2 produces the Zeitnetz's temporal structure through a circular scanning process:

1. **Concatenate** all 12 rhythm permutations into a single 144-element "rhythm tape"
2. **Concatenate** all 12 pitch permutations into a 144-element "pitch target" sequence
3. For each target pitch in sequence: scan the rhythm tape **circularly forward** from the current cursor position, counting steps until a match is found
4. The **step count** becomes the **duration** of that note in 32nd-note units

The cursor position persists between scans — it does not reset. This circular reading creates the distinctive duration patterns that define the Zeitnetz's rhythmic profile.

3.2 Example: Row 0 (Voice 1)

```
Rest: 11 thirty-second notes
cis(6)  h(9)  c(7)  gis(6)  a(6)  dis(4)  fis(3)  e(10)  d(6)  ais(3)  f(1)  g(10)
```

Each row contains exactly 12 notes. The durations vary from 1 to 17 thirty-second notes, determined entirely by the circular scanning distances.

4. Stage 3 — Klangfamilien (Sound Families)

4.1 Stage 3.1 — Starting Pitches

Function: `run_stage3()` — **Export:** `zeitnetz_stage3_1a_rows.musicxml`, `zeitnetz_stage3_1b_families.musicxml`

The 75 sound families are derived by reading the rhythm permutation rows in a specific circular order: **[7, 8, 9, 10, 11, 0, 1, 2, 3, 4, 5, 6]**. For each row, pitches are read left-to-right until a "target pitch" (determined by pitch permutation 10, the "control row") is reached, inclusive.

Row	Families	Count
Row 7	F1–F10	10
Row 8	F11–F18	8
Row 9	F19–F28	10
Row 10	F29–F38	10
Row 11	F39–F41	3
Row 0	F42–F51	10
Row 1	F52	1
Row 2	F53–F57	5
Row 3	F58–F60	3
Row 4	F61–F64	4
Row 5	F65–F70	6
Row 6	F71–F75	5
Total		75

4.2 Stage 3.2 — End Pitches

Function: `run_stage3_2()` — **Export:** `zeitnetz_stage3_2.musicxml`

Each family receives an end pitch through a complementary reading process: (1) Build an "end-pitch tape" by reading the same rows in **reverse order** [6, 5, 4, 3, 2, 1, 0, 11, 10, 9, 8, 7], **reversing each row's pitches**. (2) Pair start pitch *i* with end-tape pitch *i*.

This yields 75 families, each defined by a `(start_pc, end_pc)` pair. Notable special cases: **Families 8, 12, 34, 42, 45, 64, and 68** have identical start and end pitches (e.g., F8: gis→gis). These require special handling in Stage 4.

5. Stage 4 — Full Score

Function: `export_stage4_score()` — **Export:** `zeitnetz_stage4_score.musicxml`

This is the central stage: the complete Zeitnetz time-grid with all 75 sound families placed on a 13-staff score.

5.1 Cyclic Zeitnetz Extension

The original 12 rows of the Zeitnetz (from Stage 2) contain only 144 events — insufficient to activate all 75 families through sequential scanning. Lachenmann's solution: **cyclically extend** the Zeitnetz. After the original 12 rows (Row 0–Row 11), the rows restart as "Row 0b", "Row 1b", "Row 2b", etc., preserving the same durations but without initial rests (continuous concatenation).

The pipeline adds cyclic extensions until all 75 families have activated and deactivated. With the default inputs, **3 extra cycles** are required, producing **576 total Zeitnetz events** (144 original + 3 × 144 cyclic), **3,947**

thirty-second-note grid positions, and **329 bars** in 3/8 time.

5.2 Sequential Scan — Family Activation

A single left-to-right scan of all Zeitnetz events determines when each family activates and deactivates:

1. **Queue:** Families are queued in order F1→F75
2. **Activation:** The next queued family activates when its `start_pc` matches the current Zeitnetz event's pitch class
3. **Event reception:** Every currently active family receives every Zeitnetz event
4. **Deactivation:** A family deactivates (inclusive) when its `end_pc` matches the current event

Same-pitch deactivation rule: For families where `start_pc == end_pc` (F8, F12, F34, F42, F45, F64, F68), the family activates on the first occurrence and deactivates on the **next** occurrence of that same pitch — it must receive at least 2 events before deactivation.

5.3 Staff Assignment

Families are assigned to 12 staves below the Zeitnetz using round-robin: **Family N → Staff ((N − 1) mod 12) + 1**. This distributes families evenly: Staves 1–3 receive 7 families each; Staves 4–12 receive 6 each.

5.4 Results Summary

- **329 bars**, 3,947 grid positions
- **3 cyclic extensions** beyond the original 12 rows
- **75/75 families** activated via sequential scan
- Family spans range from F2 (2 events, pos 39→45) to F75 (18 events, pos 3172→3258)

6. Zeitnetz Version 2 — Variable Time Signatures

Function: `export_zeitnetz_v2()` — **Export:** `zeitnetz_v2.musicxml`

6.1 Concept

Every bar from Version 1 (uniform 3/8) is transformed by assigning one of 7 time signature types. The crucial principle: **each bar retains exactly 12 internal grid positions**, but the basic rhythmic unit changes proportionally to the bar's total duration.

6.2 The Seven Time Signature Types

Type	Time Sig.	Unit (bar / 12)	Notation
1	3/8	1/32 (thirty-second note)	Standard — unchanged from V1
2	4/8	1/24 (sixteenth-note triplet)	Tuplet: 3:2 sixteenths per beat
3	3/4	1/16 (sixteenth note)	Standard — double V1 durations
4	4/4	1/12 (eighth-note triplet)	Tuplet: 3:2 eighths per beat
5	3/2	1/8 (eighth note)	Standard
6	4/2	1/6 (quarter-note triplet)	Tuplet: 3:2 quarters per beat
7	12/4	1/4 (quarter note)	Standard — 8× V1 durations

6.3 The 105-Bar Cyclic Sequence

The time signatures follow a 105-bar pattern applied cyclically to all 329 bars:

```
7,6,6,5,5,5,4,4,4,4,3,3,3,3,3,2,2,2,2,2,2,
1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,
2,2,2,2,2,2,2,3,3,3,3,3,3,3,4,4,4,4,4,4,4,
5,5,5,5,5,5,5,6,6,6,6,6,6,6,
7,7,7,7,7,7,7,7,7,7,7,7,7,7,
6,6,6,6,6,6,5,5,5,5,5,4,4,4,4,3,3,3,2,2,1
```

The pattern has a characteristic **wedge/arch shape**: (1) **Descent** (bars 1–35): From type 7 (slowest) down to type 1 (fastest), with increasing repetitions as types get smaller. (2) **Ascent** (bars 36–84): Back up from type 1 to type 7. (3) **Descent** (bars 85–105): A shorter, accelerating contraction back to type 1.

6.4 Tuplet Implementation

For types 2, 4, and 6, each bar's 12 grid positions are grouped into **4 groups of 3** (one per beat). Each group receives explicit tuplet brackets (3:2 ratio). When an entire group consists of rests, it is written as a standard beat-length rest without tuplet markup.

6.5 Results Summary

- **329 bars** (same count as V1, but with varied durations)
- **13 staves** (same structure as Stage 4)
- Bar type distribution: Type 1: 45, Type 2: 45, Type 3: 49, Type 4: 49, Type 5: 48, Type 6: 47, Type 7: 46
- Time signatures written only when they change

7. Zeitnetz Final — Duration as Count

Function: `export_zeitnetz_final()` — **Export:** `zeitnetz_final.musicxml`

7.1 Concept

This is Lachenmann's final transformation of the time-grid, as described by Pena:

> *"Lachenmann's final step in generating the time-grid consists in multiplying each sound family. The durations of the tones in each family are used to COUNT NOTES in the time-grid in terms of pitches; that is, for example, what previously lasted 6 thirty-second notes now corresponds to the duration of 6 notes of the time-grid."*

7.2 Algorithm

1. **Retrieve** the family's entries from the Stage 4 sequential scan, including the Zeitnetz event index and duration of each entry
2. **First entry** remains at its original Zeitnetz event index
3. **Subsequent entries:** from the current event index, advance forward by the previous entry's duration value (counting Zeitnetz events in staff 1)
4. **Pitch:** each entry takes the pitch of the Zeitnetz event it lands on (NOT the original Stage 4 pitch)

Concrete example (Family 1):

```
Stage 4 durations: [6, 9, 7, 6, 6, 4, 3, 10]
Entry 1.1: Zeitnetz event index 0 (position 11, pitch cis)
Entry 1.2: index 0 + 6 = 6
Entry 1.3: index 6 + 9 = 15
Entry 1.4: index 15 + 7 = 22
...and so on
```

7.3 Greedy Staff Assignment

Unlike Stage 4's fixed round-robin assignment, the final version uses a **greedy interval scheduler** to minimise the number of staves: (1) Compute each family's time span. (2) Sort families by start position. (3) For each family, assign it to the first staff whose last family ends before this family starts. (4) If no staff has room, create a new one. This is the classic interval scheduling algorithm guaranteeing the minimum number of staves for non-overlapping spans.

7.4 Event Labelling

Every note in the family staves carries a lyric in the format **"F.E"** where F is the family number and E is the event number within that family (1-based). For example: "1.1" for the first event of Family 1, "35.6" for the sixth event of Family 35.

7.5 Results Summary

- **14 total staves** (1 Zeitnetz + 13 family staves — only 1 more than Stage 4)
- **313 bars**
- **75/75 families** placed, all entries computed
- Maximum grid position: 3,745
- The greedy scheduler packed 75 families into just 13 staves
- Variable time signatures from the V2 cyclic sequence applied throughout
- **The output matches the final pages of Pena's Diplomarbeit**, confirming the correctness of the entire pipeline

7.6 Family Spread Comparison

Family	Stage 4 Span	Final Span	Entries
F1	pos 11 → 62	pos 11 → 334	9
F7	pos 300 → 381	pos 300 → 883	14
F38	pos 1632 → 1757	pos 1632 → 2478	15
F75	pos 3172 → 3258	pos 3172 → 3745	18

8. Output Files

File	Stage	Staves	Bars	Description
zeitnetz_stage1.musicxml	1	12	2 each	Pitch & rhythm permutations
zeitnetz_stage2.musicxml	2	1	~83	Zeitnetz V1 (12 rows, 3/8)
zeitnetz_stage3_1a_rows.musicxml	3.1	1	12	Klangfamilien by row order
zeitnetz_stage3_1b_families.musicxml	3.1	1	12	Klangfamilien by family order
zeitnetz_stage3_2.musicxml	3.2	1	8	75 start→end bi-chords
zeitnetz_stage4_score.musicxml	4	13	329	Full score, uniform 3/8
zeitnetz_v2.musicxml	V2	13	329	Variable time signatures
zeitnetz_final.musicxml	Final	14	313	Duration-as-count, variable TS

9. Technical Architecture

9.1 Single-File Pipeline

The entire system is contained in `zeitnetz_pipeline.py`, structured as: (1) Shared pitch utilities — conversion tables, parsers. (2) Stage functions — `run_stage1()` through `run_stage5()`, plus `run_stage3_2()`. (3) Export functions — one per output file. (4) Console print functions — formatted summaries per stage. (5) Main entry point — argument parsing and sequential execution.

9.2 Key Design Decisions

- **Integer arithmetic throughout.** All grid positions are computed as integers (32nd-note units). MusicXML offsets use Python's `Fraction` type to avoid floating-point drift. This was essential — early implementations suffered from cumulative rounding errors over 329+ bars.
- **Manual measure construction.** Rather than relying on music21's automatic measure filling (which introduced positioning errors), all measures are constructed manually with explicit `m.insert(offset, element)` calls.
- **Cyclic extension with termination test.** The Zeitnetz is extended one cycle at a time, with a full sequential scan test (`_test_all_families_done()`) after each cycle. A safety cap of 10 cycles prevents infinite loops.
- **Self-contained export functions.** Each export function regenerates its own data from the Stage 2 and Stage 3.2 inputs, rather than depending on mutable shared state.

9.3 Algorithms Summary

Algorithm	Stage	Purpose
Permutation matrix construction	1	Generate 12×12 transformation matrix
Onset address computation	1	Map duration list to matrix addresses
Circular forward scanning	2	Derive durations from pitch distances
Target-based row reading	3.1	Extract 75 family starting pitches
Reverse complementary pairing	3.2	Assign end pitches to families
Sequential activation scan	4	Activate/deactivate families over Zeitnetz
Same-pitch deactivation rule	4	Handle families with identical start/end PC
Cyclic Zeitnetz extension	4, Final	Extend time-grid until all families complete
Tuplet grouping (3 per beat)	V2, Final	Proper bracket notation for triplet bars

Algorithm	Stage	Purpose
Duration-as-count transformation	Final	Convert durations to event-skip counts
Greedy interval scheduling	Final	Minimise family staves

10. Usage

```
# Run full pipeline with default Lachenmann values
python3 zeitnetz_pipeline.py --export

# Custom pitch row (German names)
python3 zeitnetz_pipeline.py --pitches "cis h c gis a dis fis e d ais f g" --export

# Custom pitch row (integers)
python3 zeitnetz_pipeline.py --pitches "0 1 2 3 4 5 6 7 8 9 10 11" --export
```

Arguments

Argument	Default	Description
--pitches	"1 11 0 8 9 3 6 4 2 10 5 7"	12 pitch classes (integers or German names)
--perm	"1 5 0 6 2 7 11 8 3 10 4 9"	Permutation pattern (12 integers, 0-indexed)
--durations	"–11 6 9 7 6 6 4 3 10 6 3 1 10"	Duration list (13 integers; first may be negative)
--axis	pitch_row[0]	Symmetry axis pitch class 0–11
--export	off	Write MusicXML files for all stages

Appendix: Development History

This pipeline was developed across multiple collaborative sessions between Paulo de Assis and Claude (Anthropic) in March 2026:

- **Session 1 (March 17):** Stages 1–3 implemented and verified against Pena's analysis. Initial standalone scripts per stage.
- **Session 2 (March 18–19):** Consolidation into single pipeline file. Stage 4 (Netz-Familien) and Stage 5 (Final Netz) implemented following Pena's OpenMusic patch descriptions.
- **Session 3 (March 20):** Complete rewrite of Stage 4 as Full Score with cyclic Zeitnetz extension, sequential scan family activation, same-pitch deactivation rule, and round-robin staff assignment. Resolved the critical 25/75 activation problem through cyclic extension. Cleanup of outdated files.
- **Session 4 (March 21):** Zeitnetz Version 2 with variable time signatures (7 types, 105-bar cyclic sequence, tuplet notation). Zeitnetz Final with duration-as-count transformation, greedy staff assignment, and event labelling. Final verification against Pena's Diplomarbeit confirmed correctness.

Report generated: 21 March 2026
Pipeline version: zeitnetz_pipeline.py (single source, ~2270 lines)
Validation: Output matches final pages of Pena's bachelor thesis